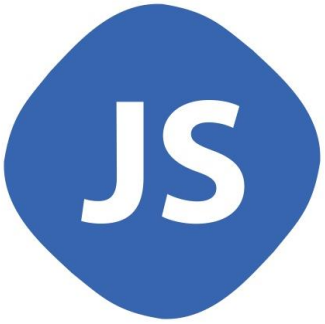


Коллекция Set

- Введение
- Коллекция Set
- Перебор коллекции Set
- Множество Set против массива

Введение

В программировании достаточно часто выполняется задача – получение уникальных элементов некоторой структуры данных.

Например, добавление в корзину заказов некоторых товаров, или составление списка посетителей. Повторное добавление товара или повторный визит не должны приводить к дубликатам. Конечная структура должна хранить только уникальные элементы.

Для решения таких и похожих задач может выступать массив хранения элементов и **дополнительный код** для проверки имеющегося элемента, например, с помощью метода **find**().

example_1. Создание массива уникальных элементов методом find()

```
<script>

    // исходный массив

    let res = ["php", "js", 1];

    // набор новых элементов для тестирования

    var newel = "js"; // не будет добавлен

    // var newel = "php "; // будет добавлен

    // var newel = 0; // будет добавлен

    // добавим в массив только уникальные значения
```



```
    if (!res.find( () => {
        // проверяем вхождение элемента в массив
        // обратите внимание на инверсию в условии
        return res.includes(newel)
    // если if вернет true запишем элемент в массив
    })) res.push(newel);
    // проверим массив
    console.log(res);
</script>
```

Или с помощью специального **цикла for...of**.

example_2. Создание массива уникальных элементов циклом for...of

```
<script>
    // исходный массив
    let res = ["php", "js", 0];
    // набор новых элементов для тестирования
    var newel = "php"; // не будет добавлен
    // var newel = "js"; // не будет добавлен
    // var newel = 20; // будет добавлен
    // добавим только уникальные значения
    for (let elem of res) {
        // выражение в скобках вернет true или false
        // обратите внимание на инверсию в условии
        if (!res.includes(newel)) {
            res.push(newel);
        }
```

```
    }

    // проверим массив
    console.log(res);
</script>
```

Но можно выполнить поставленную задачу и более элегантно. Множество Set – как раз то, что для этого нужно.

Коллекция Set

Объект Set (множество, коллекция Set) – это особый вид коллекции: **множество значений** (без ключей), где каждое значение может появляться только один раз.

> ***Важно***. **Set** — коллекция для хранения **уникальных значений любого типа**. Одно и то же значение нельзя добавить больше одного раза.

Его основные методы это:

- new **Set**(iterable) – создаёт Set, и если в качестве аргумента был предоставлен итерируемый объект (обычно это массив), то копирует его значения в новый Set.
- set.**add**(value) – добавляет значение (если оно уже есть, то ничего не делает), возвращает тот же объект set.
- set.**delete**(value) – удаляет значение, возвращает true, если value было в множестве на момент вызова, иначе false.
- set.**has**(value) – возвращает true, если значение присутствует в множестве, иначе false.
- set.**clear**() – удаляет все имеющиеся значения.
- set.**size** – возвращает количество элементов в множестве.

Основная изюминка – это то, что при повторных вызовах set.**add**() с одним и тем же значением ничего не происходит, за счёт этого как раз и получается, что каждое значение появляется один раз.

> ***Важно***. **Set** — это неиндексированная коллекция, **положить** элемент в коллекцию **можно**, но **достать нельзя**. По элементам коллекции можно **итерироваться**.

В случае с примерами из **example_1** или **example_2** можно сказать, что у коллекции Set лучше производительность, потому что метод или цикл итерируют весь массив для проверки наличия элемента.

Множество Set лучше **оптимизировано** для добавлений, оно автоматически проверяет на уникальность.

example_3. Создание объекта Set

```
<script>
    // создание и инициализация коллекции строковыми значениями
    const set = new Set(['Набоков', 'Толстой', 'Чехов',
    'Чехов']);
    console.log("Разммер массива:", set.size); // 3
    // вывод коллекции в консоль
    console.log(set);
    // ха-ха
    console.log(set[0]); // так не получится
</script>
```

При работе с множеством Set обратите внимание, не бывает двух или более одинаковых объектов. Если у одного человека две руки и у другого две руки это не значит, что это одинаковые люди, это разные личности, то есть **разные объекты**.



В следующем демонстрационном примере в коллекции Set будет три объекта, потому что первый и третий объекты **похожи, но не одинаковы**. Можно сравнивать только **значения** объектов, а не **сами объекты**.

example_4. Создание множества объектов

```
<script>
    // создание и инициализация коллекции объектами
    let set = new Set(
        [
            // первый и третий элементы похожи, но не одинаковы
            { name: "John" },
            { name: "Pete" },
            { name: "John" }
        ]
    );
    // размер коллекции
    console.log(set.size);
    // вывод коллекции в консоль
    console.log(set);
</script>
```

Объекты, добавленные в Set-коллекцию, могут быть настолько сложными, насколько этого требует предметная область решаемой задачи.

example_5. Создание множества сложных объектов

```
<script>
    // объекты коллекции могут быть сколько угодно сложными
    let set = new Set(
        [
```



```
            {id: 1, name: "JavaScript разработчик с нуля",
            leader: 1},
            {id: 2, name: "Python-рузработчик", leader: 2},
            {id: 3, name: "Data Scientist", leader: 2},
            {id: 4, name: "Фронтенд-разработчик на JavaScript",
            leader: 3}
        ]
    );
    // вывод коллекции в консоль
    console.log(set);
</script>
```

В демонстрационном примере **example_6** протестируем метод **add**() для работы с элементами-объектами коллекции.

example_6. Метод add() объекта Set

```
<script>
    // инициируем коллекцию
    let set = new Set(
        [
            {id: 1, name: "JavaScript разработчик с нуля",
            leader: 1},
            {id: 2, name: "Python-рузработчик", leader: 2},
            {id: 3, name: "Data Scientist", leader: 2},
            {id: 4, name: "Фронтенд-разработчик на JavaScript",
            leader: 3}
        ]
    );
```

```
    console.log(set.size); // 4
    // добавим в коллекцию новые элементы
    set.add( { id: 5, name: "Тестирование веб-приложений на Python", leader: 1} );
    set.add( { id: 6, name: "Разработчик C++", leader: 2} );
    console.log(set.size); // 6
    // добавить можно только один элемент
    set.add(
        {id: 7, name: "Android-разработчик", leader: 1},
        {id: 8, name: "HTML, CSS, JavaScript для чайников", leader: 3},
        {id: 9, name: "Программирование на Java", leader: 4}
    );
    console.log(set.size); // 7
    console.log(set);
</script>
```

Коллекция Set может хранить **разные типы** элементов.

example_7. Коллекция может хранить разные типы элементов

```
<script>
    // инициируем коллекцию строковыми значениями
    let set = new Set(
        [
            "JavaScript разработчик с нуля",
            "Python-рузработчик",
            "Data Scientist"
        ]
```

```
    );
    // добавим в коллекцию массив
    set.add( [4, "Фронтенд-разработчик на JavaScript", 3] );
    // добавим в коллекцию объект
    set.add( { id: 6, name: "Разработчик C++", leader: 2} );
    // вывод коллекции в консоль
    console.log(set);
</script>
```

В первых двух демонстрационных примерах я показал, каким образом можно получить уникальные значения массива. Будет логично, если последним примером я повторю эту операцию, используя коллекцию Set (множество Set).

example_8. Создание коллекции уникальных элементов

```
<script>
    // исходный массив
    let res = ["php", "js", 0];
    // пустая коллекция Set
    let set = new Set();
    // набор новых элементов для тестирования
    var newel = "php"; // не будет добавлен
    // var newel = "js "; // будет добавлен
    // var newel = 20; // будет добавлен
    res.push(newel);
    // коллекция сохранит только уникальные значения
    for (let elem of res) {
        set.add(elem);
    };
```

```
    // проверим значения коллекции
    console.log(set);
</script>
```

Все достаточно просто, логично, ну а главное – никаких инверсий ;)

Рассмотрим способы перебора элементов коллекции.

Перебор коллекции Set

Коллекция Set имеет те же встроенные методы, что и коллекция Map:

- set.**keys**() – возвращает перебираемый объект для значений.
- set.**values**() – то же самое, что и set.**keys**(), присутствует для обратной совместимости с **Map**.
- set.**entries**() – возвращает перебираемый объект для пар вида [значение, значение], присутствует для обратной совместимости с **Map**.

В следующем примере обратите внимание на значения, получаемые методами **keys**() и **values**(). Сравните со значениями методов коллекции **Map**.

example_9. Методы keys() и values() коллекции Set

```
<script>
    // инициируем коллекцию строковыми значениями
    let set = new Set(
        [
            "JavaScript разработчик с нуля",
            "Python-рузработчик",
            "Data Scientist",
            "Фронтенд-разработчик на JavaScript",
            "Тестирование веб-приложений на Python",
```

```
            "Разработчик C++"
        ]
    );
    console.log("Это объект keys:", set.keys());
    console.log("Это объект values:", set.values());
</script>
```

И еще раз обратите внимание на вывод в консоль результата метода **entries**().

example_10. Метод entries() коллекции Set

```
<script>
    // инициируем коллекцию значениями-объектами
    let set = new Set(
        [
            {id: 1, name: "JavaScript разработчик с нуля",
            leader: 1},
            {id: 2, name: "Python-рузработчик", leader: 2},
            {id: 3, name: "Data Scientist", leader: 2},
            {id: 4, name: "Фронтенд-разработчик на JavaScript",
            leader: 3},
            {id: 5, name: "Тестирование веб-приложений на
            Python", leader: 1},
            {id: 6, name: "Разработчик C++", leader: 2},
            {id: 7, name: "Android-разработчик", leader: 1},
            {id: 8, name: "HTML, CSS, JavaScript для чайников",
            leader: 3},
            {id: 9, name: "Программирование на Java", leader: 4}
        ]
```

```
    );
    console.log("Это объект entries:", set.entries());
</script>
```

Мы можем перебрать содержимое объекта Set как с помощью специального цикла **for..of**, так и используя метод **forEach**.

Перебор множества специальным циклом for...of

example_11. Перебор коллекции Set циклом for...of

```
<script>
    // инициируем коллекцию значениями-объектами
    let set = new Set(
    [
        {id: 1, name: "JavaScript разработчик с нуля", leader:
        1},
        {id: 2, name: "Python-рузработчик", leader: 2},
        {id: 3, name: "Data Scientist", leader: 2},
        {id: 4, name: "Фронтенд-разработчик на JavaScript",
        leader: 3},
        {id: 5, name: "Тестирование веб-приложений на Python",
        leader: 1},
        {id: 6, name: "Разработчик C++", leader: 2},
        {id: 7, name: "Android-разработчик", leader: 1},
        {id: 8, name: "HTML, CSS, JavaScript для чайников",
        leader: 3},
        {id: 9, name: "Программирование на Java", leader: 4}
    ]);
```

```
    // итерируем коллекцию специальным циклом for
    for (let course of set) {
    console.log(course);
    }
</script>
```

Перебор множества методом forEach

example_12. Перебор коллекции Set методом forEach

```
<script>
    // инициируем коллекцию значениями-объектами
    let set = new Set(
    [
        {id: 1, name: "JavaScript разработчик с нуля", leader:
        1},
        {id: 2, name: "Python-разработчик", leader: 2},
        {id: 3, name: "Data Scientist", leader: 2},
        {id: 4, name: "Фронтенд-разработчик на JavaScript",
        leader: 3},
        {id: 5, name: "Тестирование веб-приложений на Python",
        leader: 1},
        {id: 6, name: "Разработчик C++", leader: 2},
        {id: 7, name: "Android-разработчик", leader: 1},
        {id: 8, name: "HTML, CSS, JavaScript для чайников",
        leader: 3},
        {id: 9, name: "Программирование на Java", leader: 4}
    ]);
```

```
    // итерируем коллекцию методом forEach
    set.forEach ( (item, key) => {
        // выводим пару ключ-значение
        console.log("item: ", item);
        console.log("key: ", key);
        console.log("----------");
    });
</script>
```

Множество Set против массива

Большинство начинающих разработчиков используют, как правило, базовые типы JavaScript:

- **number**
- **string**
- **object**
- **array**
- **boolean**.

Для большинства случаев этого вполне достаточно. Но если вы хотите сделать код максимально быстрым и масштабируемым – базовых типов может не хватить. Коллекция **Map** и множество **Set** могут сделать ваш код быстрее, особенно при работе с большими массивами данных.

Между коллекциями и массивами много общего, но внедрение коллекций придает вашему приложению скорость выполнения сценария, что сложно достигнуть с массивами.

Разница между Set и массивом

Самое важное отличие множества от массива в том, что массив — это индексированная коллекция. То есть элементы массива отсортированы по их индексам.

С другой стороны, коллекции Set не имеют индексов. Их элементы итерируемы в порядке их добавления и должны быть уникальными.

Преимущества Set

В прямом сравнении коллекции Set имеют несколько преимуществ перед массивами, особенно если говорит о скорости выполнения:

Поиск частей. использование indexOf() или includes() для поиска элемента в массиве работают сравнительно медленно.

Удаление частей. В Set элемент удаляется по значению. В массиве используем эквивалент splice() на основе индекса элемента. Как и в предыдущем пункте, получаем замедление из-за зависимости от индексов.

Вставка частей. Быстрее добавить элемент в коллекцию, чем делать это в массив через push(), unshift() или другим способом.

Удаление дубликатов. Все элементы уже уникальны.

Сложность выполнения. Методы поиска элементов массива имеют линейную сложность выполнения, это означает сложность растет по мере роста количества элементов. У коллекций размер не влияет на скорость выполнения.

В **самостоятельных работах** к уроку рассмотрим:

- Создание массива уникальных значений.
- Создание коллекции из связанных массивов.
- Создание коллекции с элементами-массивами.
- Создание коллекции с элементами-объектами.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы